TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

BÀI TẬP LỚN

MÔN HỌC

HỆ THỐNG NHÚNG

Sinh viên: Vũ Lân

MSV: k225480106036

Lớp: K58KTP.K01

Giáo viên hướng dẫn: TS. Nguyễn Văn Huy

Thái Nguyên – 2026

TRƯỜNG ĐHKTCN **CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM**

KHOA ĐIỆN TỬ **Độc lập - Tự do - Hạnh phúc**

BÀI TẬP LỚN

MÔN HỌC HỆ THỐNG NHÚNG

BỘ MÔN CÔNG NGHỆ THÔNG TIN

Sinh viên: Vũ Lân

Lớp: K58KMT.K01 **Ngành: Kỹ thuật máy tính**

Giáo viên hướng dẫn: TS. Nguyễn Văn Huy

Ngày giao đề: 19/01/2026 **Ngày hoàn thành: 22/03/2026**

Tên đề tài: Đồng Hồ Dương Lịch

Yêu cầu: - Tự động đọc giờ từ ic thời gian thực, đọc và tính toán nhiệt độ từ cảm biến nhiệt độ và hiển thị lên các led 7 đoạn.

- Cho phép người dùng thay đổi thời gian qua bàn phím.

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....
.....
.....
.....
.....
.....
.....

Thái Nguyên, ngày…tháng …năm2026

Giáo viên hướng dẫn

(ký và ghi rõ họ tên)

MỤC LỤC

HÌNH ẢNH & BẢNG BIỂU 4
LỜI NÓI ĐẦU 5
CHƯƠNG 1: TỔNG QUAN HỆ THỐNG 6
1.1 KHẢO SÁT VÀ PHÂN TÍCH ĐỀ BÀI 6
1.2 LỰA CHỌN GIẢI PHÁP 7
1.2.1 Giải pháp công nghệ 7
1.2.2 Giải pháp thiết kế 8
1.2.3 Xác định bài toán và giới hạn của đề tài 8
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG 9
2.1 Sơ đồ khối tổng quát của hệ thống 9
2.2 Sơ đồ call Graph 10
2.3 Sơ đồ đặc tả của hệ thống 10
2.4 Các module trong hệ thống 11
2.4.1 Module điều khiển, reset, setup. 11
2.4.2 Module hiển thị 11
2.4.3 Module tạo thời gian thực 11
2.4.4 Module xử lý 11
2.4.5 Module nối tiếp 12
2.4.6 Module cảm biến nhiệt độ 12
2.5 Lựa chọn linh kiện 12
2.5.1 Vi điều khiển PIC16F877A 12
2.5.2 Led 7 thanh 16
2.5.3. IC thời gian thực DS1307 18
2.5.4 Nút bấm(button) 20
2.5.5 IC 74HC595 21
CHƯƠNG 3: CHƯƠNG TRÌNH HỆ THỐNG 24
3.1 Code main.h 24
3.2 Code main.c 24
CHƯƠNG 4: MÔ PHỎNG MẠCH 47
4.1 Mô phỏng mạch 47
CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ 49
5.1 Kiểm thử hệ thống 49
5.2 Đánh giá hệ thống 49

HÌNH ẢNH & BẢNG BIỂU

Hình 2.1: Sơ đồ tổng quát hệ thống đồng hồ thời gian thực. 9
Hình 2.2: Sơ đồ call graph 10
Hình 2.3: Sơ đồ đặc tả hệ thống thời gian thực 10
Hình 2.5: PIC16F877A 12
Hình 2.6: Sơ đồ PIC16F877A 15
Hình 2.7: sơ đồ chân 7 seg-com-anode và hình ảnh minh họa 17
Hình 2.8: Sơ đồ vị trí các led 17
Hình 2.9: DS1307 20
Hình 2.10 : Nút bấm trong thực tế và trong Proteus. 21
Hình 2.11: Nguyên lý hoạt động của nút bấm. 21
Hình 2.12: Sơ đồ chân 74HC595 22
Hình 4.1: Mạch mô phỏng hệ thống đồng hồ dương lịch 47
Hình 4.2: Mạch mô phỏng hệ thống đồng hồ dương lịch chạy 48

BẢNG BIỂU

Bảng 2.1: Bảng mã của led Anode chung 18
Bảng 2.2 : Sơ đồ chân 74HC595 23

LỜI NÓI ĐẦU

Từ khi thế giới tiến vào kỷ nguyên mới, kỷ nguyên của khoa học công nghệ nói chung và kỹ thuật điều khiển nói riêng. Hệ thống điều khiển tự động có ứng dụng rộng rãi trong các lĩnh vực như: kỹ thuật điện tử, viễn thông, điều khiển tự động… Trong hoạt động thường ngày các thiết bị điện tử điều khiển tự động ngày càng được du nhập một cách nhanh chóng. Các thiết bị điện tử đang dần thay thế mọi hoạt động của con người. Từ đó sinh ra một ngành mới mang tên gọi là "hệ thống nhúng". Từ khi ra đời đến nay các hệ thống đóng góp vai trò hết sức quan trọng trong các hệ thống điều khiển.

Sau gần 4 năm học tập và thực hành tại trường, em đã dược làm quen với các môn học chuyên nhành. Với mục đích hiểu rõ, nắm bắt công nghê ngành vi sử lý vi điều khiển và khai thác ứng dụng của nó trong cuộc sống, để gắn liền lý thuyết và thực tế thấy được những tính năng ưu việt của dòng vi xử lý vi điều khiển ta đi tìm hiểu rõ đề tài: **"đồng hồ dương lịch".**

Do kiến thức chuyên môn vẫn còn hạn chế, tài liệu tham khảo còn nhiều chưa thực sự tiếp cận được nhiều, trong quá trình hoàn thiện bài còn nhiều sai sót. Em mong thầy cô có thể xem xét và bổ xung ý kiến để phần bài tập lớn của em được hoàn thiện hơn, em có thể dùng những ý khiến bổ xung đó làm nền tảng trong việc học tập tại và tương lai.

Em xin chân thành cảm ơn!

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG

1.1 KHẢO SÁT VÀ PHÂN TÍCH ĐỀ BÀI

❖ Giới thiệu về hệ thống thời gian thực

Trong lĩnh vực công nghệ thông tin, người ta nói về hệ thống thông tin thời gian thực khi hệ thống đó điều khiển một vật thể vật lý với một tốc độ phù hợp với sự tiến triển của tiến trình chủ. Một vị dụ dễ hiểu (hệ thống thông tin điều khiển màn hình hiển thị giờ chính xác của các tàu điện ngầm sẽ đến và đi tại một gare nhất định). Hệ thống thông tin thời gian thực khác với những hệ thống thông tin khác bởi sự gò bó về thời gian, do đó việc tuân thủ các nguyên tắc cũng quan trọng như độ chính xác của kết quả, nói một cách khác, hệ thống không chỉ đơn giản là đưa ra kết quả chính xác mà nó còn phải thực hiện một xử lý trong một thời gian rất ngắn. Hệ thống thông tin thời gian thực ngày nay được ứng dụng trong rất nhiều lĩnh vực như trong ngành công nghiệp sản xuất, kiểm soát tiến trình (trong nhà máy, hay trong viện hạt nhân, trong hệ thống hàng không, thông qua các hệ thống dẫn đường tích hợp trên máy bay và vệ tinh). Sự phát triển của hệ thống thông tin thời gian thực yêu câu mỗi phân từ của hệ thống phải ở thời gian thực.

❖ Khái niệm về hệ thống thời gian thực

Một hệ thời gian thực (RTC) là một hệ thống mà sư hoạt động tin cậy của nó không chỉ phụ thuộc vào sự chính xác của kết quả, mà còn phụ thuộc vào thời điểm đưa ra kết quả, hệ thống có lỗi khi yêu cầu về về thời gian không được thoả mãn. Hệ thống thời gian thực thiết kể nhằm cho phép trả lời lại các yếu tố kích thích phát sinh từ các thiết bị phần cứng trong một ràng buộc thời gian xác định. Ở đây ta có thể hiểu thế nào là một RTS (real time systems) bằng cách hiểu thế nào là một tiến trình, một công nghệ thời gian thực. Nhìn chung, trong những RTS chỉ có một số công việc được gọi là công việc thời gian thực, các công việc này có một mức độ khẩn cấp riêng phải hoàn tất. Sự thay đổi của sự kiện trong thế giới thực xảy ra rất nhanh, mỗi tiến trình giám sát sự kiện này phải thực hiện việc xử lý trong một khoảng thời gian ràng buộc gọi là deadline, khoảng thời gian ràng buộc này được xác định bởi thời gian bắt đầu và thời gian hoàn tất công việc. Trong thực tế, các yêu

tố kích thích này xảy ra trong thời gian rất ngắn vào khoảng vài mili giây, thời gian mà hệ thống trả lại yêu tố kích thích đó tốt nhất vào khoảng dưới một giây, thường vào khoảng vài chục mili giây, khoảng thời gian này bao gồm thời gian tiếp nhận kích thích, xử lý thông tin và trả lời kích thích. Một yếu tố khác cần quan tâm trong RTS là những công việc thời gian thực này có tuần hoàn hay không? Công việc tuần hoàn thì ràng buộc thời gian ấn định trong từng chu kỳ xác định, công việc không tuần hoàn xảy ra với ràng buộc thời gian vào lúc bắt đầu và kết thúc công việc, ràng buộc này chỉ được xác định vào lúc bắt đầu công việc. Các biển cổ kích hoạt công việc không tuần hoàn thường dựa trên kỹ thuật xử lý ngắt của hệ thống phần cứng.

Về cấu tạo, RTS thường được cấu thành từ các thành tổ chính sau:

- Đồng hồ thời gian thực: cung cấp thông tin thời gian thực.
- Bộ điều khiển ngắt: quản lý các biến cố khong theo chu kỳ.
- Bộ định hiểu: quản lý các quá trình thực hiện.
- Bộ điều khiển thực hiện: khởi động các quá trình.

Các thành tố trên có thể phân định là thành phần cứng hay phần mềm tùy thuộc vào hệ thống và ý nghĩa sử dụng. Thông thường các RTS được kết hợp vào phần cứng có khả năng tốt hơn so với phần mềm có chức năng tương ứng và tránh được chi phí quá đắt cho việc tối ưu hóa phần mềm. Ngày nay chi phí cho phần cứng ngày càng rẻ, chọn lựa ưu tiên phần cứng là một xu hướng chung.

1.2 LỰA CHỌN GIẢI PHÁP

1.2.1 Giải pháp công nghệ

Qua phân tích trên em xây dựng hệ thống đồng hồ thời gian thực hiển thị thời gian: năm, tháng, ngày, giời, phút, giây và hiển thị nhiệt độ theo thời gian thực qua led 7 đoạn.

Thời gian cập nhật tự động và hiển thị qua led 7 đoạn có thể quan sát trên màn hình mô phỏng.

1.2.2 Giải pháp thiết kế

- Đầu vào hệ thống lấy dữ liệu từ DS1307, là ic thời gian thực nhằm cung cấp giờ cho hệ thống để cập nhật thời gian thực năm, tháng, ngày, giời, phút, giây.
- Để hiển thị dùng led 7 thanh: lấy tín hiệu ra từ vi điều khiển thông báo thời gian thực năm, tháng, ngày, giờ, phút, giây, nhiệt độ tính được.
- Xử lý, điều khiển dùng PIC16F877A. Lập trình vi điều khiển để xử lý đọc ghi giờ, tính toán thời gian.
- Dùng module 74HC595 làm chân nối giữa PIC16F877A với led 7 thanh.
- Điều khiển thời gian thông qua nút bấm(button): Lựa chọn cài đặt giờ, tang giảm thời gian.

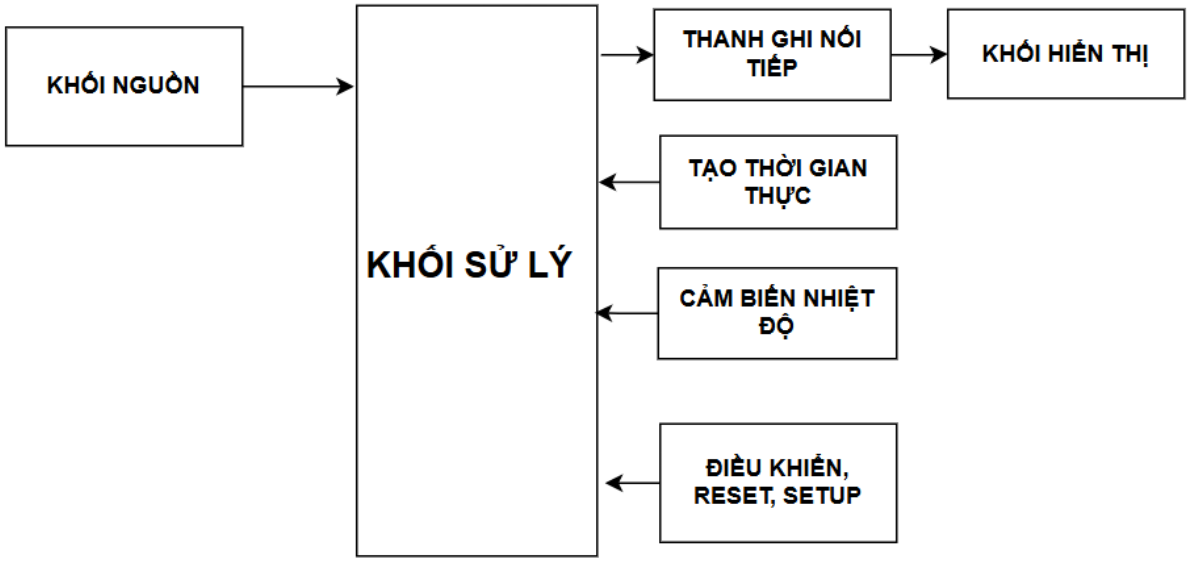
1.2.3 Xác định bài toán và giới hạn của đề tài

Hệ thống đồng hồ dương lịch:

- Đầu vào hệ thống lấy dữ liệu từ DS1307, là IC thời gian thực nhằm cung cấp giờ cho hệ thống. Đầu ra được hiển thi qua led 7 thanh.
- Hiển thị chính xác thời gian thực, năm, tháng, ngày, giờ, phút, giây dương lịch.
- Chỉnh được thời gian qua BUTTON.
- Làm việc với điện áp cấp từ pin.
- Làm việc ổn định và lâu dài.
- Hiển thị nhiệt độ lên led 7 thanh theo nhiệt độ tính toán ra.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

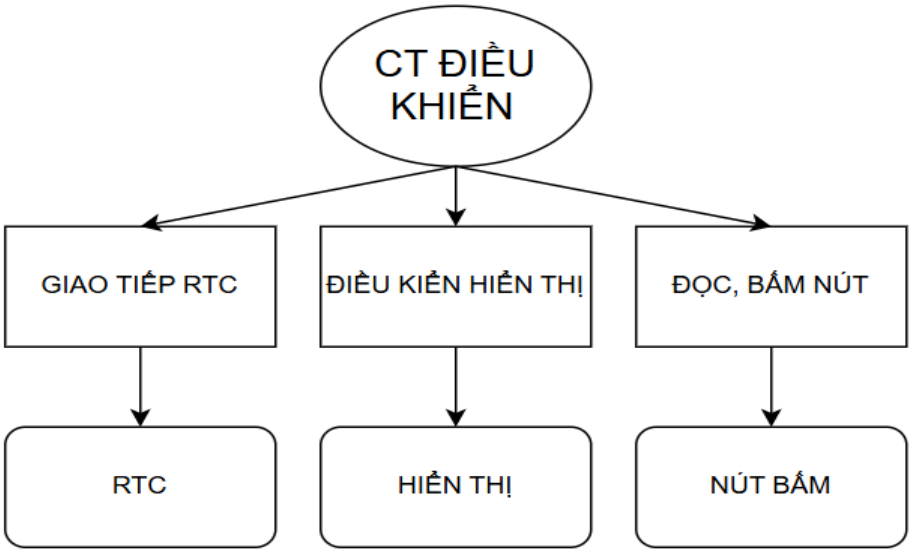
2.1 Sơ đồ khối tổng quát của hệ thống



Hình 2.1: Sơ đồ tổng quát hệ thống đồng hồ thời gian thực.

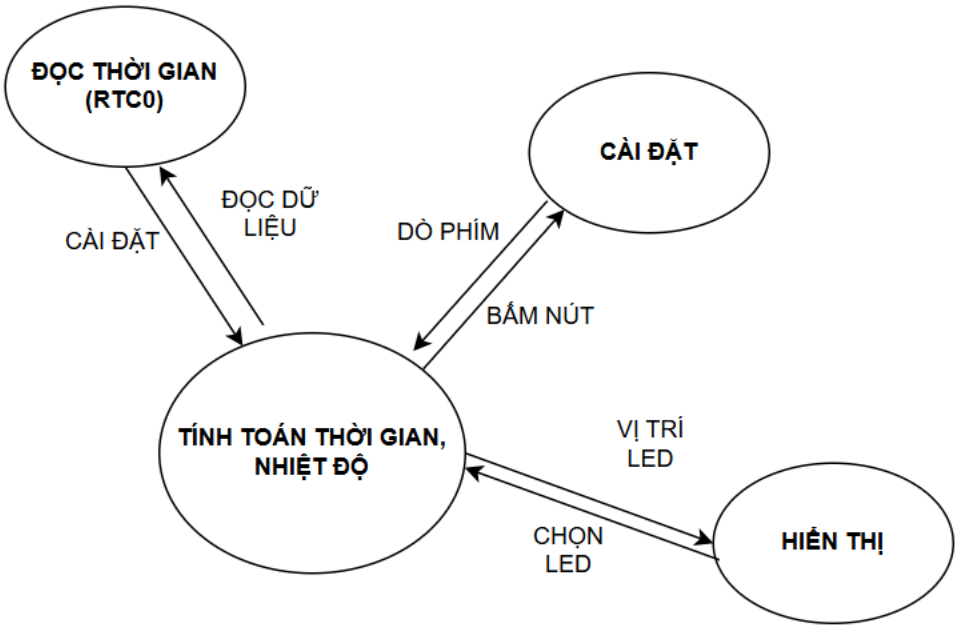
- Khối nguồn: Cung cấp nguồn cho hệ thống.
- Khối hiển thị: lấy tín hiệu ra từ từ chân pic qua thanh ghi nối tiếp để hiển thị thời gian, năm, tháng, ngày, giờ, phút, giây, nhiệt độ được tính toán.
- Khối điều khiển, reset, setup: gồm có 12 nút bấm có tác dụng điều chỉnh thời gian, năm, tháng, ngày, giời, phút, giây trong mạch và 1 nút để tùy chỉnh cài đặt, 1 nút dùng để hủy bỏ mọi thay đổi đưa về như lúc đầu.
- Khối tạo thời gian thực: cung cấp thông tin về năm, tháng, ngày, giờ, phút, giây.
- Khối xử lý: dùng VDK để lấy ví tín hiệu từ khối điều khiển, tạo thời gian thực, tính toán nhiệt độ đưa ra khối hiển thị.

2.2 Sơ đồ call Graph



Hình 2.2: Sơ đồ call graph

2.3 Sơ đồ đặc tả của hệ thống



Hình 2.3: Sơ đồ đặc tả hệ thống thời gian thực

2.4 Các module trong hệ thống

2.4.1 Module điều khiển, reset, setup.

- Module dùng để điều khiển hệ thống theo yêu cầu:

+Thiết lập được trong vi điều khiển là làm việc bình thường hay chuyển sang chế độ cài đặt thời gian.

+Điều chỉnh tang -giảm thời gian, năm, tháng, ngày…Kết thúc việc thiết lập thời gian và điều chỉnh thời gian bằng một nút điều khiển.

+Ta lựa chọn bộ điều khiển 12 nút bấm.

- Khối reset có tác dụng đưa vi điều khiển về trạng thái ban đầu .
- Khối setup có tác dụng lưu mọi cài đặt thay đổi vừa thiết lập.

2.4.2 Module hiển thị

- Khối hiển thị: lấy tín hiệu ra từ chân IC để hiển thị thời gian thực năm, tháng, ngày, giờ, phút, giây và nhiệt độ theo tính toán.
- Lựa chọn hiển thị qua led 7 thanh, màu đỏ.

2.4.3 Module tạo thời gian thực

-IC thời gian thực cần hoạt động được với nguồn cung cấp nhỏ, dùng đề cập nhật thời gian và ngày tháng. Cung cấp thông tin về giờ, phút giây thứ, ngày, tháng, năm Ngày cuối tháng sẽ tự động được điều chỉnh với các tháng nhỏ hơn 31 ngày bao gồm cả việc tự động nhảy năm. Đồng hồ có thể hoạt động ở dạng 24h hoặc 12h với chỉ thị AM PM.

-Khi mất nguồn IC thời gian thực vẫn duy trỳ hoạt động không cần điều chỉnh lại thời gian. IC có thêm nguồn riêng từ pin 3V.

-Với điều kiện như trên ta lựa chọn IC thời gian thực là IC DS1307.

2.4.4 Module xử lý

- Dùng VDK đẻ lấy tín hiệu từ khối điều khiển, tạo thời gian thực, nhiệt độ, đưa ra khối hiển thị. Yêu cầu:

+Tốc độ xử lý nhanh, chính xác.

+Bộ nhớ không cần lớn, cũng không quá nhỏ.

=> Lựa chọn vi điều khiển PIC16F877A.

2.4.5 Module nối tiếp

- Dùng để tạo khúc nối giữa các chân của VDK và chân của led do khối lượng chân của led cần dùng tới rất nhiều mà chân của VDK có hạn.
- Dùng IC 74HC595 làm module nối tiếp.

2.4.6 Module cảm biến nhiệt độ

- IC cảm biến nhiệt độ hoạt động được với nguồn cung cấp nhỏ, dùng để cập nhật nhiệt độ sau khi qua quá trình tính toán để hiển thị lên led hiển thị.
- Phù hợp với yêu cầu ta lựa chọn IC LM35.

2.5 Lựa chọn linh kiện

2.5.1 Vi điều khiển PIC16F877A

a) Cấu tạo và chức năng các khối của PIC16F877A.

Hình 2.5: PIC16F877A

_Cấu trúc tổng quát của pic 16f877a:

+) 8K x 14 words Flash ROM.

+)368 x 8 Bytes RAM.

+) 256 x 8 Bytes EEPROM.

+) 5 Port xuất/nhập (A, B, C, D, E) tương ứng 33 chân.

+) 2 Bộ định thời 8 bit Timer 0 và Timer 2.

+) 1 Bộ định thời 16 bit Timer 1, có thể hoạt động ở chế độ tiết kiệm năng lượng (SLEEP MODE) với nguồn xung clock ngoài.

+) 2 Bộ CCP Capture/Compare/PWM. (Bắt Giữ/ So Sánh/ Điều Biến Xung)

+) 1 Bộ biến đổi Analog to Digital 10 bit, 8 ngõ vào.

- Có 35 bộ lệnh
- Hoạt động với tần số lên đến 20Mhz
- Điện áp hoạt động từ 4.2V đến 5V. Nếu cấp điện áp vượt quá 5V thì sẽ bị hư.
- Không có bộ dao động bên trong giống như PIC18F46K22, PIC18F4550.
- Dòng điện tối đa trên mỗi PORT khoảng 100mA. Do vậy, dòng điện tối đa cho mỗi chân GPIO của PIC 16F877A là 10mA
- Có 4 package: PDIP 40 chân, PLCC 44 chân, TQFP 44 chân, QFN 44 chân

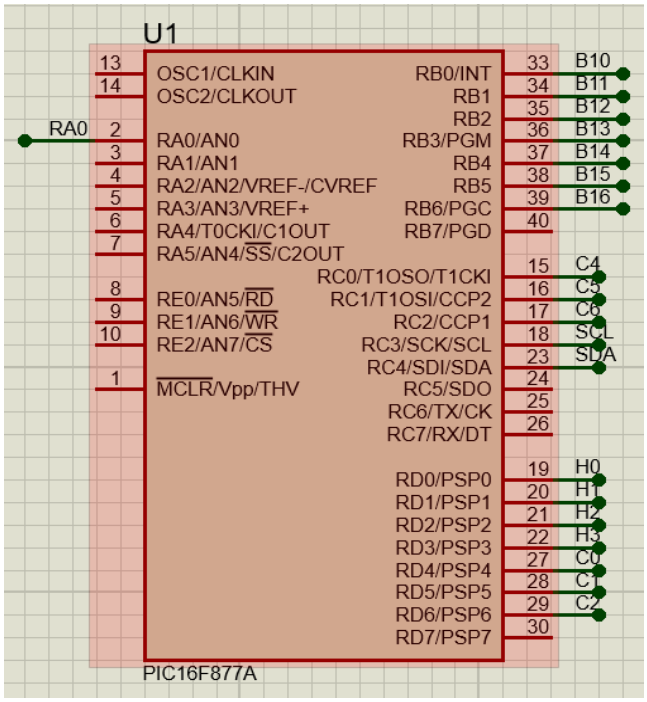
b) Chức năng của các chân PIC16F877A.

- Chân 1: MCLR là chân clear của mạch này. Nó sẽ khởi động lại vi điều khiển và được kích hoạt bởi mức logic thấp, có nghĩa là chân này phải được cấp liên tục một điện áp 5V và nếu cấp điện áp 0V thì PIC16F877A sẽ bị đặt lại.
- Một nút nhấn và một điện trở được kết nối đến chân này. Chân MCLR này luôn được cấp điện áp 5V. Khi muốn khởi động lại mạch. Bạn chỉ cần nhấn vào nút nhấn thì chân MCLR sẽ được đưa về 0 và mạch được đặt lại.
- Chân 2 RA0/AN0: PORT A có 6 chân, từ chân số 2 đến chân số 7. Tất cả đều là các chân xuất, nhập dữ liệu hai chiều. Chân số 2 là chân đầu tiên của PORT A. Chân này có thể được sử dụng như một chân tương tự (analog) chân AN0. Nó được tích hợp bộ chuyển đổi analog sang digital.

- Chân 3 RA1/AN1: Đầu vào tín hiệu analog 1
- Chân 4 RA2/AN2/Vref-: Có thể hoạt động như đầu vào analog thứ 2 hoặc chân điện áp tham chiếu âm.
- Chân 5 RA3/AN3/Vref+: Có thể hoạt động như đầu vào analog thứ 3 hoặc chân điện áp tham chiếu dương.
- Chân 6 RA0/T0CKI: Với timer 0, chân này hoạt động được như một đầu vào xung clock và đầu ra open drain.
- Chân 7 RA5/SS/AN4: Có thể hoạt động như một đầu vào analog thứ 4. Có cổng nối tiếp đồng bộ và là chân SS cho cổng này.
- Chân 8 RE0/RD/AN5: PORT E bắt đầu từ chân số 8 đến chân số 10 và là cổng I/O hai chiều. Nó còn là cổng analog thứ 5 hoặc là chân RD (tích cực mức logic thấp) cho cổng slave giao tiếp song song
- Chân 9 RE1/WR/AN6: Là đầu vào analog thứ 6 và là chân WR (tích cực mức logic thấp) cho cổng slave giao tiếp song song.
- Chân 10 RE2/CS/A7: Là đầu vào analog 7 và là chân CS cho cổng slave song song.
- Chân 11 và 32 VDD: Đây là hai chân cấp nguồn 5V.
- Chân 12 và 31 VSS: Các chân tham chiếu nối đất cho I/O và các chân logic. Chúng nên được nối với 0V hoặc mắc GND.
- Chân 13 OSC1/CLKIN: Là đầu vào bộ dao động hoặc chân đầu vào xung nhịp bên ngoài.
- Chân 14 OSC2/CLKOUT: Đây là chân đầu ra của bộ dao động. Một bộ dao động thạch anh được nối vào giữa hai chân 13 và 14 để cấp xung nhịp bên ngoài cho bộ vi điều khiển. ¼ tần số của OSC1 được OSC2 xuất ra trong chế độ RC. Điều này xác định tốc độ chu kỳ xử lý lệnh.
- Chân 15 RC0/T1OCO/T1CKI: PORT C có 8 chân. Là cổng I/O hai chiều. Trong số đó, chân 15 là chân đầu tiên. Nó có thể là đầu vào xung nhịp của bộ định thời 1 hoặc đầu ra bộ dao động của bộ định thời 2.
- Chân 16 RC11/T1OSI/CCP2: Là đầu vào dao động của bộ định thời 1 hoặc đầu vào capture 2 / đầu ra so sánh 2 / đầu ra PWM 2.

- Chân 17 RC2/CCP1: Đầu vào capture 1/ đầu ra so sánh 1/ đầu ra PWM1
- Chân 18 RC3/SCK/SCL: Đầu ra của chế độ SPI hoặc I2C và có thể là I/O cho bộ dao động nối tiếp đồng bộ.
- Chân 23 RC4/SDI/SDA: Chân dữ liệu trong chế độ SPI hoặc là chân xuất nhập dữ liệu chế độ I2C.
- Chân 24 RC5/SDO: Là chân xuất dữ liệu chế độ SPI.
- Chân 25 RC6/TX/CK: Có thể là chân xung clock đồng bộ hoặc chân truyền không đồng bộ UART.
- Chân 26 RC7/RX/DT: Là chân dữ liệu đồng bộ hoặc chân nhận tín hiệu UART.
- Các chân 19, 20, 21, 22, 27, 28, 29, 30: Tất cả các chân này đều thuộc PORT D, đây là một cổng I/O hai chiều. Khi bus vi xử lý được kết nối, nó có thể hoạt động ncho cổng slave giao tiếp dữ liệu song song.
- Chân 33-40 PORT B: Hai chân này đều thuộc PORTB. Trong đó RB0 có thể được sử dụng làm chân ngắt ngoài và RB6 và RB7 có thể được sử dụng làm chân debugger.

C) Sơ đồ PIC16F877A trong mạch.



Hình 2.6: Sơ đồ PIC16F877A

2.5.2 Led 7 thanh

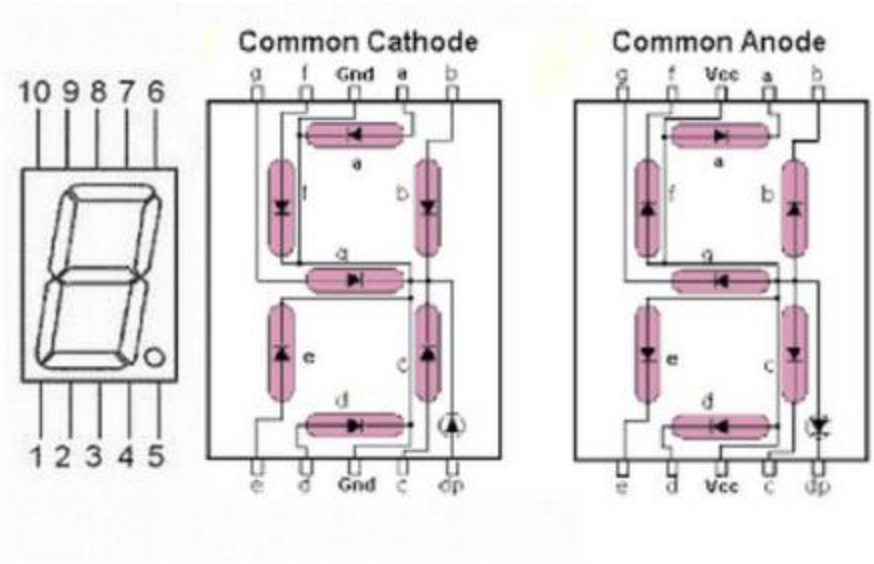
a) Các khái niệm cơ bản:

Trong các thiết bị, để bảo trạng thái hoạt động của thiết bị đó cho người sử dụng với thông số chỉ là các dãy số đơn thuần, thường người ta sử dụng "led 7 đoạn". Led 7 đoạn được sử dụng khi các đầy số không đòi hỏi quá phức tạp, chỉ cần hiện thị số là đủ, chẳng hạn led 7 đoạn được dùng để hiển thị nhiệt độ phòng, trong các đồng hồ treo tường bằng điện tử, hiển thị số lượng sản phẩm được kiểm tra sau một công đoạn nào đó..

Led 7 đoạn có cấu tạo bao gồm 7 led đơn có dạng thanh xếp theo hìnhvà có thêm. ột led đơn hình tròn nhỏ thể hiện dấu chấm tròn ở góc dưới, bên phải của led 7 đoạn.

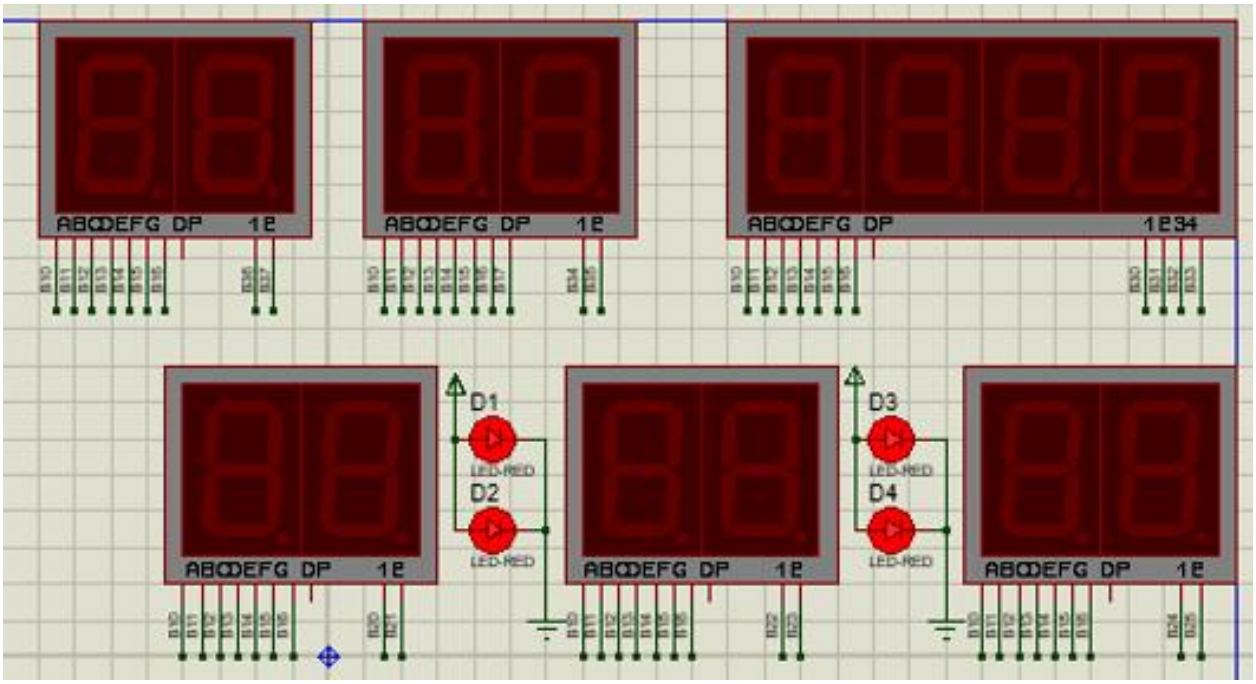
7 led đơn trên led 7 đoạn có Anode (cực -) hoặc Cathode (cực -) được nối chung vớinhau vào một điểm, được đưa chân ra ngoài để kết nối với mạch điện. 8 cực còn lại trênmỗi led đơn được đưa thành 8 chân riêng, cũng được đưa ra ngoài để kết nối với mạchđiện. Nêu led 7 đoạn có Anode (cực +) chung, đầu chung này được nối với +Vcc, cácchân còn lại dùng để điều khiển trạng thái sáng tắt của các led đơn, led chỉ sảng khi tỉnhiệu đặt vào các chân này ở mức 0. Nêu led 7 đoạn có Cathode (cực -) chung, đầu chungnày được nỗi xuống Ground (hay Mass), các chân còn lại dùng để điều khiển trạng thái sáng tắt của các led đơn, led chỉ sáng khi tín hiệu đặt vào các chân này ở mức 1.

Vì led 7 đoạn chứa bên trong nó các led đơn, do đó khi kết nổi cần đảm bảo dòng qua mỗi led đơn trong khoảng 10mA-20mA để bảo vệ led. Nếu kết nổi với nguồn 5V cóthể hạn dòng bằng điện trở 3300 trước các chân nhận tin hiệu điều khiển.



Hình 2.7: sơ đồ chân 7 seg-com-anode và hình ảnh minh họa

b) Sơ đồ vị trí các led:



Hình 2.8: Sơ đồ vị trí các led

C) Kết nối với vi điều khiển:

Ngõ nhận tín hiệu điều khiển của led 7 đoạn ra có 8 đường, vì vậy có thể dùng 1 port nào đó cảu vi điều khiển để điều khiển led 7 đoạn. Vì số led 7 thanh cần dùng nhiều lên ta

sử dụng module nối 74HC595 để nối ra nhiều chân hơn. Như vậy led 7 thanh nhận một dữ liệu 8 bit từ vi điều khiển để điều khiển hoạt động sáng tắt của từng led đơn trong nó, dữ liệu được xuất ra điều khiển mã hiển thị lè 7 thanh thường được gọi là " mã hiển thị led 7 thanh". Có hai kiểu mac hiển thị led 7 đoạn : mã dành cho led 7 đoạn có anode (cực +) chung và mã dành cho led 7 thanh có cathode(cực -) chung. Chẳng hạn , để hiển thị số 1 cần các led vị trí b và c điện áp là 0V(mức 0) các chân còn lại được đặt điện áp là 5V(mức 1), nếu sử dụng led 7 thanh cso cathode chung thì ngược lại.

Phần cứng được nối với 1 port bất kì của vi điều khiển , để thuận tiện cho việc sử lí về sau phần cứng nên dược kết nối như sau: Px.0 nối với chân a, Px.1 nối với chân b, lần lượt theo thứ tự cho đến Px.7 nối với chân h.

d) Bảng mã của led Anode chung

- Bảng mã cho led Anode chung (a là LSB, dp là MSB)

Số	dp	g	f	e	d	c	b	a	Mã hex
0	1	1	0	0	0	0	0	0	C0h
1	1	1	1	1	1	0	0	1	F9h
2	1	0	1	0	0	1	0	0	A4h
3	1	0	1	1	0	0	0	0	B0h
4	1	0	0	1	1	0	0	1	99h
5	1	0	0	1	0	0	1	0	92h
6	1	0	0	0	0	0	1	0	82h
7	1	1	1	1	1	0	0	0	F8h
8	1	0	0	0	0	0	0	0	80h
9	1	0	0	1	0	0	0	0	90h

Bảng 2.1: Bảng mã của led Anode chung

2.5.3. IC thời gian thực DS1307

a) Giới thiệu chung về DS13070

- IC thời gian thực là họ vì điều khiển của hãng Dalat. DS1307 có một số đặc | trưng cơ bản sau:

+ DS1307 là IC thời gian thực với nguồn cung cấp nhỏ dùng để cập nhật thời gian và ngày tháng, năm.

+SRAM 56bytes.

+ Địa chỉ và dữ liệu được truyền nối tiệp qua 2 đường bus 2 chiều.

+ DS1307 có một mạch cảm biến điện áp dùng để dò các điện áp lỗi và tự động đóng ngắt với nguồn pin cung cấp 3V.

+ DS1307 có 7 byte dữ liệu nằm từ địa chỉ 0x00 tới 0x06, 1byte điều khiển và 56 byte lưu trữ (dành cho người sử dụng).

+ Khi xử lý dữ liệu từ DS1307, họ đã tự chuyển cho ta về dạng số BCD, vi dụ như ta đọc được dữ liệu từ địa chỉ 0x04 (tương ứng với Day- ngày trong tháng) và tại 0x05 (tháng) là 0x15, 0x11.

+ Lưu ý đến vai trò của chân SQW/OUT. Đây là chân cho xung ra của DS1307 có 4 chế độ 1Hz, 4.096HZ, 8.192Hz, 32.768Hz...các chế độ này được quy định bởi các bít của thanh ghi Control Register (địa chỉ 0x07).

+ Địa chỉ của DS1307là 0xD0.

-Cơ chế hoạt động: DS1307 hoạt động với vai trò slave trên đường bus nổi tiếp. Việc truy cập được thi hành với chỉ thị start và một mã thiết bị nhất định được cung cấp bởi địa chỉ các thanh ghi. Tiếp theo đó các thanh ghi sẽ được truy cập liên tục đến khi chỉ thị stop được thực thi.

b) Cơ chế hoạt động và chức năng của DS1307:

Vcc: nối với nguồn

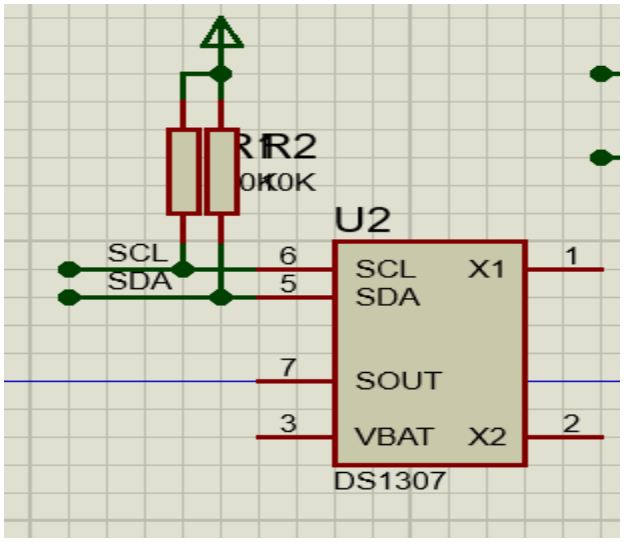
Vbat: đầu vào pin 3V

GND: đất

SDA: chuỗi data

SCL: dây xung clock

SQW OUT: xung vuông/đầu ra driver



Hình 2.9: DS1307

- DS1307 là một IC thời gian thực với nguồn cung cấp nhỏ, dùng đề cập nhật thời gian và ngày tháng với 56 bytes SRAM. Địa chỉ và dữ liệu được truyên nồi tiếp qua 2 đường bus 2 chiều. Nó cung cấp thông tin về giờ, phút giây thứ ngày tháng, năm Ngày cuối tháng sẽ tự động được điều chỉnh với các tháng nhỏ hơn 31 ngày bao gồm cả việc tự động nhày năm. Đồng hồ có thể hoạt động ở dạng 24h hoặc 12h với chỉ thị AM PM. DS1307 có một mạch cảm biến điện áp dùng để đò các điện áp lỗi và tự động đóng ngắt với nguồn pin cung cấp.
- DS 1307 hoạt động với vai trò slave trên đường bus nôi tiếp. Việc truy cập được thi hành với chỉ thị START và một mã thiết bị nhất định được cung cấp bởi địa chỉ các thanh ghi. Tiếp theo đó các thanh ghi sẽ được truy cập liên tục đên khi chỉ thị STOP được thực thi.

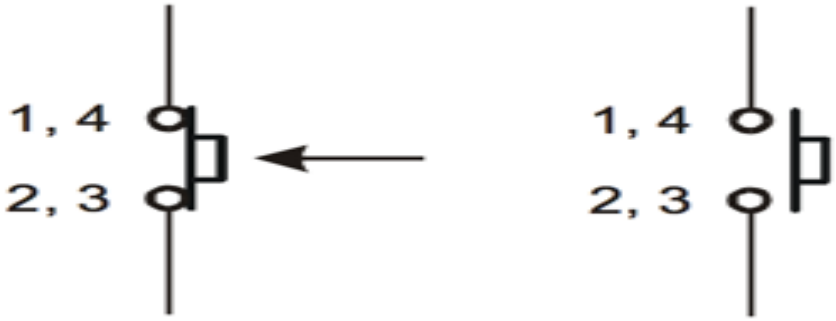
2.5.4 Nút bấm(button)

Nút ấn là một loại công tắc đơn giản điều khiển hoạt động của máy hoặc một số loại quá trình. Hầu hết, các nút nhấn là nhựa hoặc kim loại. Hình dạng của nút ấn có thể phù hợp với ngón tay hoặc bàn tay để sử dụng dễ dàng. Tất cả phụ thuộc vào thiết kế cá nhân. Nút ấn có 2 loại chính là nút nhấn thường mở hoặc nút nhấn thường đóng.

Hình 2.10 : Nút bấm trong thực tế và trong Proteus.

Nguyên lí làm việc của nút nhấn :

Nút nhấn có ba phần: Bộ truyền động, các tiếp điểm cố định và các rãnh. Bộ truyền động sẽ đi qua toàn bộ công tắc và vào một xy lanh mỏng ở phía dưới. Bên trong là một tiếp điểm động và lò xo. Khi nhấn nút, nó chạm vào các tiếp điểm tĩnh làm thay đổi trạng thái của tiếp điểm. Trong một số trường hợp, người dùng cần giữ nút hoặc nhấn liên tục để thiết bị hoạt động. Với các nút nhấn khác, chốt sẽ giữ nút bật cho đến khi người dùng nhấn nút lần nữa.



Hình 2.11: Nguyên lý hoạt động của nút bấm.

2.5.5 IC 74HC595

IC 74HC595 là một IC thanh ghi dịch có 16 chân bao gồm một chân chốt loại D cùng với một thanh ghi dịch bên trong chip. Nó nhận dữ liệu đầu vào nối tiếp và sau đó gửi dữ liệu này ra ngoài thông qua các chân song song.

Ngoài các đầu ra song song, nó còn cung cấp một đầu ra nối tiếp, nó có các đầu vào xung nhịp riêng lẻ cho thanh ghi dịch và chân chốt D. IC này thuộc họ linh kiện logic HC được thiết kế để sử dụng trong các ứng dụng công nghệ CMOS.

74HC595 có hai thanh ghi tích hợp. Cái đầu tiên là thanh ghi dịch và cái thứ hai là thanh ghi lưu trữ. Dữ liệu được truyền nối tiếp để thay đổi từng bit thanh ghi. Nhưng nó chỉ chuyển vào thanh ghi lưu trữ khi chân chốt dữ liệu được kích ở mức cao.

Sơ đồ chân của 74HC595

Hình 2.12: Sơ đồ chân 74HC595.

Số chân	Tên chân	Mô tả
1, 2, 3, 4, 5, 6, 7	Chân output (Q1 đến Q7)	74hc595 có 8 chân đầu ra, trong đó có 7 chân này. Chúng có thể được kiểm soát nối tiếp
8	Ground	Nối đất
9	(Q7') Serial Output	Chân này được sử dụng để kết nối nhiều hơn một 74hc595 dưới dạng xếp tầng
10	(MR) Master Reset	Reset tất cả các đầu ra ở mức thấp. Phải giữ ở mức cao để hoạt động bình thường
11	(SH_CP) Clock	Đây là chân đồng hồ mà tín hiệu đồng hồ phải được cung cấp từ vi điều khiển hoặc vi xử lý
12	(ST_CP) Latch	Chân Latch dùng để cập nhật dữ liệu vào các chân đầu ra. Nó kích hoạt mức cao
13	(OE) Output Enable	Chân OE được sử dụng để tắt đầu ra. Phải giữ ở mức thấp để hoạt động bình thường
14	(DS) Serial Data	Đây là chân mà dữ liệu được gửi đến, dựa trên đó 8 đầu ra được điều khiển
15	(Q0) Output	Chân đầu ra đầu tiên
16	Vcc	Chân này cấp nguồn cho IC, thường sử dụng + 5V

Bảng 2.2 : Sơ đồ chân 74HC595

CHƯƠNG 3: CHƯƠNG TRÌNH HỆ THỐNG

3.1 Code main.h

```
#include <16F877A.h>

#device adc=8

#FUSES NOWDT                    //No Watch Dog Timer

#FUSES HS                       //High speed Osc (> 4mhz for PCM/PCH) (>10mhz for PCD)

#FUSES NOPUT                    //No Power Up Timer

#FUSES NOPROTECT                //Code not protected from reading

#FUSES NODEBUG                  //No Debug mode for ICD

#FUSES NOBROWNOUT               //No brownout reset

#FUSES NOLVP                    //No low voltage prgming, B3(PIC16) or B5(PIC18) used
for I/O

#FUSES NOCPD                    //No EE protection

#FUSES NOWRT                    //Program memory not write protected

#FUSES RESERVED                 //Used to set the reserved FUSE bits

#use delay(clock=20000000)
```

3.2 Code main.c

```
#include <DS1307.C>
#fuses HS, NOWDT, NOPROTECT, NOLVP
#device ADC=10
#use delay(clock=20M)
#use I2C(MASTER, I2C1, SLOW = 100000, STREAM = DS1307_STREAM)
#include <16F877A.h>
#define DATA_PIN_CHOT   PIN_C0
#define CLOCK_PIN_CHOT  PIN_C1
#define LATCH_PIN_CHOT  PIN_C2
#define LED_COUNT  8
```

```
RTC_Time *mytime;
unsigned int8 maphim,k,vonglap_1,vonglap_2;
signed int16 C;
unsigned int16 Gio,Phut,Giay,Ngay,Thang,Nam;
unsigned int16 Hour, Minute, Second, Day, Month, Year, Year_2;
unsigned int16 Nam1, Nam2, Thang1, Thang2, Ngay1, Ngay2, Gio1, Gio2, Phut1, Phut2,
Giay1, Giay2;
unsigned char data1=0x00,data2=0x00,maled[10] = {
  0b11000000,  // 0
  0b11111001,  // 1
  0b10100100,  // 2
  0b10110000,  // 3
  0b10011001,  // 4
  0b10010010,  // 5
  0b10000010,  // 6
  0b11111000,  // 7
  0b10000000,  // 8
  0b10010000   // 9
};
const unsigned int8 phim[]={1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
void CHOT(unsigned char data1, unsigned char data2) {
  for (k = 0; k < 8; k++) {
    output_bit(DATA_PIN_CHOT, (data1 >> (7 - k)) & 1);
    output_high(CLOCK_PIN_CHOT);
    output_low(CLOCK_PIN_CHOT);
  }
  for (k = 0; k < 8; k++) {
    output_bit(DATA_PIN_CHOT, (data2 >> (7 - k)) & 1);
    output_high(CLOCK_PIN_CHOT);
    output_low(CLOCK_PIN_CHOT);
  }
  output_high(LATCH_PIN_CHOT);
  output_low(LATCH_PIN_CHOT);
```

```
}

unsigned int8 quet_phim()
{
  unsigned int8 hang, mp=0xff;
  unsigned int8 maquet[4]={0xFE, 0xFD, 0xFB, 0xF7};
  for(hang=0; hang<=3; hang++)
  {
    output_D(maquet[hang]);
    while(input(pin_D4)==0) mp=hang*3+1;
    while(input(pin_D5)==0) mp=hang*3+2;
    while(input(pin_D6)==0) mp=hang*3+3;
  }
  return mp;
}

void hienthi_main()
{
   output_B(maled[Gio/10%10]);
   data1=0x01;
   CHOT(data2,data1);
   delay_ms(1);
   data1=0x00;
   CHOT(data2,data1);
   output_B(maled[Gio%10]);
   data1=0x02;
   CHOT(data2,data1);
   delay_ms(1);
   data1=0x00;
   CHOT(data2,data1);
   output_B(maled[Phut/10%10]);
   data1=0x04;
```

```
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[Phut%10]);
data1=0x08;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[Giay/10%10]);
data1=0x10;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[Giay%10]);
data1=0x20;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[(C/ 10) % 10]);
data1=0x40;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[C%10]);
data1=0x80;
CHOT(data2,data1);
delay_ms(1);
```

```
data1=0x00;
CHOT(data2,data1);

output_B(maled[2]);
data2=0x01;
CHOT(data2,data1);
delay_ms(1);
data2=0x00;
CHOT(data2,data1);
output_B(maled[0]);
data2=0x02;
CHOT(data2,data1);
delay_ms(1);
data2=0x00;
CHOT(data2,data1);
output_B(maled[Nam/10%10]);
data2=0x04;
CHOT(data2,data1);
delay_ms(1);
data2=0x00;
CHOT(data2,data1);
output_B(maled[Nam%10]);
data2=0x08;
CHOT(data2,data1);
delay_ms(1);
data2=0x00;
CHOT(data2,data1);

output_B(maled[Thang/10%10]);
data2=0x10;
CHOT(data2,data1);
delay_ms(1);
```

```
    data2=0x00;
    CHOT(data2,data1);
    output_B(maled[Thang%10]);
    data2=0x20;
    CHOT(data2,data1);
    delay_ms(1);
    data2=0x00;
    CHOT(data2,data1);

    output_B(maled[Ngay/10%10]);
    data2=0x40;
    CHOT(data2,data1);
    delay_ms(1);
    data2=0x00;
    CHOT(data2,data1);
    output_B(maled[Ngay%10]);
    data2=0x80;
    CHOT(data2,data1);
    delay_ms(1);
    data2=0x00;
    CHOT(data2,data1);
}

void hienthi()
{
    output_B(maled[Gio1]);
    data1=0x01;
    CHOT(data2,data1);
    delay_ms(1);
    data1=0x00;
    CHOT(data2,data1);
    output_B(maled[Gio2]);
```

```
data1=0x02;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[Phut1]);
data1=0x04;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[Phut2]);
data1=0x08;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[Giay1]);
data1=0x10;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[Giay2]);
data1=0x20;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[(C/ 10) % 10]);
data1=0x40;
CHOT(data2,data1);
```

```
delay_ms(1);
data1=0x00;
CHOT(data2,data1);
output_B(maled[C%10]);
data1=0x80;
CHOT(data2,data1);
delay_ms(1);
data1=0x00;
CHOT(data2,data1);

output_B(maled[2]);
data2=0x01;
CHOT(data2,data1);
delay_ms(1);
data2=0x00;
CHOT(data2,data1);
output_B(maled[0]);
data2=0x02;
CHOT(data2,data1);
delay_ms(1);
data2=0x00;
CHOT(data2,data1);
output_B(maled[Nam1]);
data2=0x04;
CHOT(data2,data1);
delay_ms(1);
data2=0x00;
CHOT(data2,data1);
output_B(maled[Nam2]);
data2=0x08;
CHOT(data2,data1);
delay_ms(1);
```

```
    data2=0x00;
    CHOT(data2,data1);

    output_B(maled[Thang1]);
    data2=0x10;
    CHOT(data2,data1);
    delay_ms(1);
    data2=0x00;
    CHOT(data2,data1);
    output_B(maled[Thang2]);
    data2=0x20;
    CHOT(data2,data1);
    delay_ms(1);
    data2=0x00;
    CHOT(data2,data1);

    output_B(maled[Ngay1]);
    data2=0x40;
    CHOT(data2,data1);
    delay_ms(1);
    data2=0x00;
    CHOT(data2,data1);
    output_B(maled[Ngay2]);
    data2=0x80;
    CHOT(data2,data1);
    delay_ms(1);
    data2=0x00;
    CHOT(data2,data1);
}

void main() {
  set_tris_C(0x18);
```

```
set_tris_B(0x00);
set_tris_D(0xF0);
OSC_Enable();
SQWE_Set(OUT_1Hz);
setup_adc(ADC_CLOCK_INTERNAL);
setup_adc_ports(AN0);
set_adc_channel(0);
while (true)
{
  mytime = RTC_Get();
  Gio = mytime->hours;
  Phut = mytime->minutes;
  Giay = mytime->seconds;
  Ngay = mytime->day;
  Thang = mytime->month;
  Nam = mytime->year;
  C = abs((read_adc() * 4.8875) / 10.0);
  hienthi_main();
  maphim=quet_phim();
  if (maphim!=0xff)
  {
      if (maphim == 12)
      {
        Nam1 = 0, Nam2 = 0, Thang1 = 0, Thang2 = 1, Ngay1 = 0, Ngay2 = 1;
        Gio1 = 0, Gio2 = 0, Phut1 = 0, Phut2 = 0, Giay1 = 0, Giay2 = 0;
        vonglap_1=1;
        vonglap_2=1;
        if (vonglap_1==1)
        {
          while (vonglap_2==1)
          {
            hienthi();
```

```
      maphim=quet_phim();
      if (maphim>=1&&maphim<=10)
      {
        Nam1=phim[maphim-1];
      }
      if (maphim==12)
      {
        vonglap_2=0;
      }
      if (maphim==11)
      {
        vonglap_2=0;
        vonglap_1=0;
      }
    }
  }
  vonglap_2=1;
  if (vonglap_1==1)
  {
    while (vonglap_2==1)
    {
      hienthi();
      maphim=quet_phim();
      if (maphim>=1&&maphim<=10)
      {
        Nam2=phim[maphim-1];
      }
      if (maphim==12)
      {
        vonglap_2=0;
        year = Nam1*10 + Nam2;
      }
```

```
      if (maphim==11)
      {
        vonglap_2=0;
        vonglap_1=0;
      }
    }
  }
  vonglap_2=1;
  if (vonglap_1==1)
  {
    while (vonglap_2==1)
    {
      hienthi();
      maphim=quet_phim();
      if (maphim==1||maphim==10)
      {
        Thang1=phim[maphim-1];
        if (Thang1==1)
        {
          Thang2=0;
        }
        else Thang2=1;
      }
      if (maphim==12)
      {
        vonglap_2=0;
      }
      if (maphim==11)
      {
        vonglap_2=0;
        vonglap_1=0;
      }
```

```
  }
}
vonglap_2=1;
if (vonglap_1==1)
{
  while (vonglap_2==1)
  {
    hienthi();
    maphim=quet_phim();

    if (Thang1==1)
    {
      if (maphim==1||maphim==2||maphim==10)
      {
        Thang2=phim[maphim-1];
      }
    }
    if (Thang1==0)
    {
      if (maphim>=1&&maphim<=9)
      {
        Thang2=phim[maphim-1];
      }
    }
    if (maphim==12)
    {
      vonglap_2=0;
      Month=Thang1*10+Thang2;
    }
    if (maphim==11)
    {
      vonglap_2=0;
```

```
        vonglap_1=0;
      }
    }
  }
  vonglap_2=1;
  if (vonglap_1==1)
  {
    while (vonglap_2==1)
    {
      hienthi();
      maphim=quet_phim();
      if (Month==2)
      {
        if (maphim==1||maphim==2||maphim==10)
        {
          Ngay1=phim[maphim-1];
        }
      }
      else
      {
        if (maphim==1||maphim==2||maphim==3||maphim==10)
        {
          Ngay1=phim[maphim-1];
        }
      }
      if (maphim==12)
      {
        vonglap_2=0;
      }
      if (maphim==11)
      {
        vonglap_2=0;
```

```
            vonglap_1=0;
          }
        }
      }
      vonglap_2=1;
      Year_2=Year+2000;
      if (vonglap_1==1)
      {
        while (vonglap_2==1)
        {
          hienthi();
          maphim=quet_phim();
          if
(Month==1||Month==3||Month==5||Month==7||Month==8||Month==10||Month==12)
          {
            if (Ngay1==3)
            {
              if (maphim==1||maphim==10)
              {
                Ngay2=phim[maphim-1];
              }
            }
            if (Ngay1==1||Ngay1==2)
            {
              if (maphim>=1&&maphim<=10)
              {
                Ngay2=phim[maphim-1];
              }
            }
            if (Ngay1==0)
            {
              if (maphim>=1&&maphim<=9)
              {
```

```
        Ngay2=phim[maphim-1];
      }
    }
  }
  if (Month==4||Month==6||Month==9||Month==11)
  {
    if (Ngay1==3) Ngay2=0;
    if (Ngay1==1||Ngay1==2)
    {
      if (maphim>=1&&maphim<=10)
      {
        Ngay2=phim[maphim-1];
      }
    }
    if (Ngay1==0)
    {
      if (maphim>=1&&maphim<=9)
      {
        Ngay2=phim[maphim-1];
      }
    }
  }
  if (Month==2)
  {
    if (Ngay1==0)
    {
      if (maphim>=1||maphim<=9)
      {
        Ngay2=phim[maphim-1];
      }
    }
    if (Ngay1==1)
```

```
    {
      if (maphim>=1&&maphim<=10)
      {
        Ngay2=phim[maphim-1];
      }
    }
    if (Ngay1==2)
    {
      if ((Year_2)%100!=0&&(Year_2)%4==0)
      {
        if (maphim>=1&&maphim<=10)
        {
          Ngay2=phim[maphim-1];
        }
      }
      else
      {
        if (maphim>=1&&maphim<=8)
        {
          Ngay2=phim[maphim-1];
        }
        if (maphim==10)
        {
          Ngay2=phim[maphim-1];
        }
      }
    }
  }

  if (maphim==12)
  {
    vonglap_2=0;
```

```
        Day=Ngay1*10+Ngay2;
      }
      if (maphim==11)
      {
        vonglap_2=0;
        vonglap_1=0;
      }
    }
  }
  //Hours
  vonglap_2=1;
  if (vonglap_1==1)
  {
    while (vonglap_2==1)
    {
      hienthi();
      maphim=quet_phim();
      if (maphim==1||maphim==2||maphim==10)
      {
        Gio1=phim[maphim-1];
      }
      if (maphim==12)
      {
        vonglap_2=0;
      }
      if (maphim==11)
      {
        vonglap_2=0;
        vonglap_1=0;
      }
    }
  }
```

```
vonglap_2=1;
if (vonglap_1==1)
{
  while (vonglap_2==1)
  {
    hienthi();
    maphim=quet_phim();
    if (Gio1==2)
    {
      if (maphim==1||maphim==2||maphim==3||maphim==10)
      {
        Gio2=phim[maphim-1];
      }
    }
    else
    {
      if (maphim>=1&&maphim<=10)
      {
        Gio2=phim[maphim-1];
      }
    }
    if (maphim==12)
    {
      vonglap_2=0;
      Hour = Gio1*10+Gio2;
    }
    if (maphim==11)
    {
      vonglap_2=0;
      vonglap_1=0;
    }
  }
```

```
}
vonglap_2=1;
if (vonglap_1==1)
{
  while (vonglap_2==1)
  {
    hienthi();
    maphim=quet_phim();
    if (maphim>=0&&maphim<=5)
    {
      Phut1=phim[maphim-1];
    }

    if (maphim==12)
    {
      vonglap_2=0;
    }
    if (maphim==11)
    {
      vonglap_2=0;
      vonglap_1=0;
    }
  }
}
vonglap_2=1;
if (vonglap_1==1)
{
  while (vonglap_2==1)
  {
    hienthi();
    maphim=quet_phim();
```

```
      if (maphim>=0&&maphim<=10)
      {
        Phut2=phim[maphim-1];
      }
      if (maphim==12)
      {
        vonglap_2=0;
        Minute = Phut1*10+Phut2;
      }
      if (maphim==11)
      {
        vonglap_2=0;
        vonglap_1=0;
      }
    }
  }
  vonglap_2=1;
  if (vonglap_1==1)
  {
    while (vonglap_2==1)
    {
      hienthi();
      maphim=quet_phim();
      if (maphim>=0&&maphim<=5)
      {
        Giay1=phim[maphim-1];
      }

      if (maphim==12)
```

```
      {
        vonglap_2=0;
      }
      if (maphim==11)
      {
        vonglap_2=0;
        vonglap_1=0;
      }
    }
  }
  vonglap_2=1;
  if (vonglap_1==1)
  {
    while (vonglap_2==1)
    {
      hienthi();
      maphim=quet_phim();
      if (maphim>=0&&maphim<=10){
        Giay2=phim[maphim-1];
      }
      if (maphim==12){
        vonglap_2=0;
        Second = Giay1*10+Giay2;
        mytime->hours = Hour;
        mytime->minutes = Minute;
        mytime->seconds = Second;
        mytime->year = Year;
        mytime->month = Month;
        mytime->day = Day;
        RTC_Set(mytime);
      }
      if (maphim==11){
```

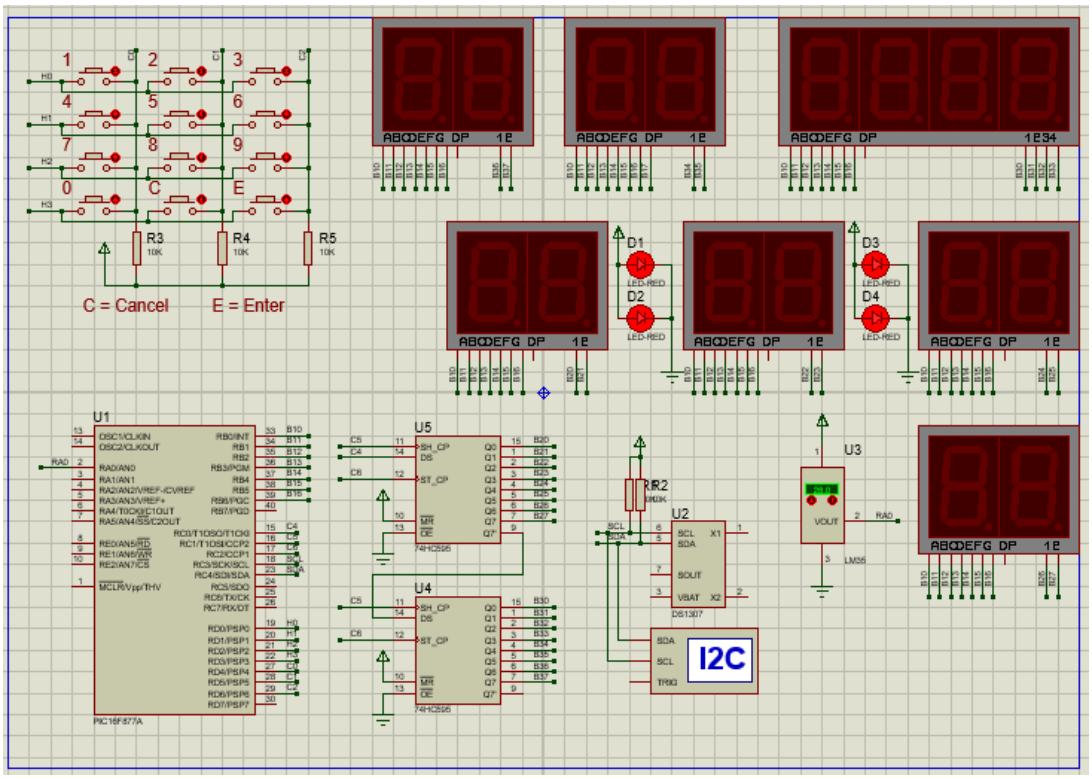
```
                vonglap_2=0;
                vonglap_1=0;
            }
        }
    }
  }
 }
}
```

CHƯƠNG 4: MÔ PHỎNG MẠCH

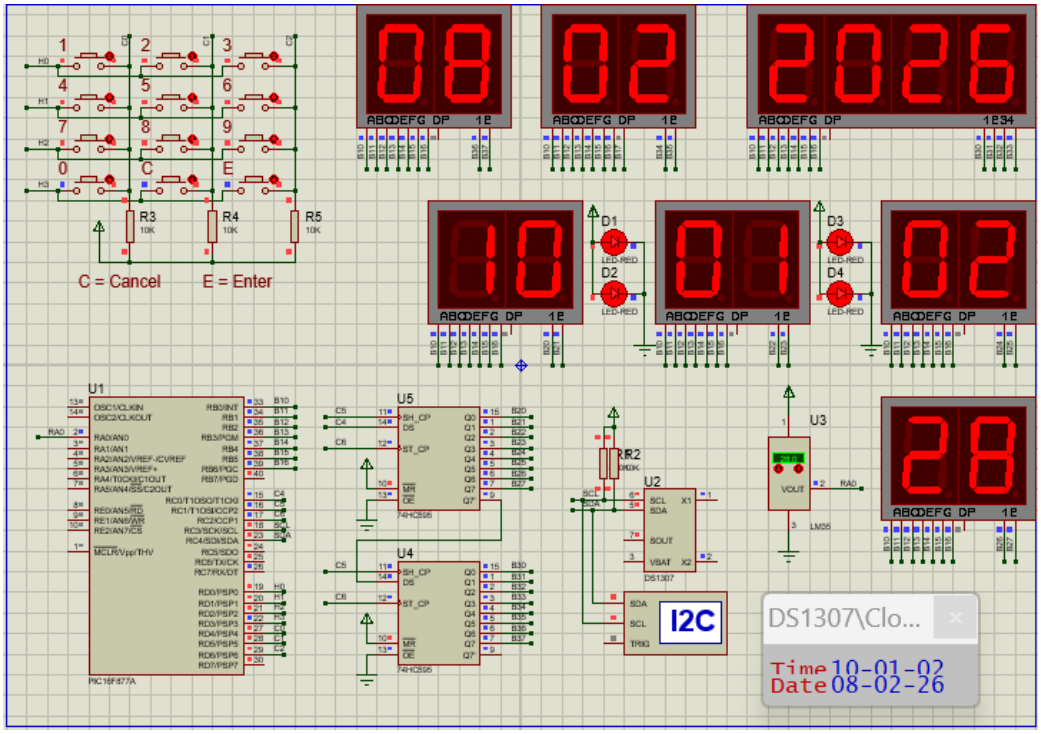
4.1 Mô phỏng mạch

Để mô phỏng mạch của hệ thống em dùng phần mềm hỗ trợ mô phỏng proteus. Hệ thống mạch led 7 thanh được thiết kế để hiển thị năm, tháng, ngày, giờ, phút, giây theo sự điều chỉnh do IC thời gian thực DS1307 và thay đổi qua nút bấm.

- Chế độ mặc định : hiển thị năm, tháng, ngày, giờ, phút, giây theo thời gian thực.
- Bấm nút E để điều chỉnh số hiển thị lên led 7 thanh mỗi lần bấm là di chuyển đến vị trí cần thay đổi sau đó bấm nút tại vị trí số để cài đặt số mong muốn.
- Hiển thị nhiệt độ tính được lên led 7 thanh



Hình 4.1: Mạch mô phỏng hệ thống đồng hồ dương lịch.



Hình 4.2: Mạch mô phỏng hệ thống đồng hồ dương lịch chạy

CHƯƠNG 5: KIỂM THỬ VÀ ĐÁNH GIÁ

5.1 Kiểm thử hệ thống

Kiểm thử các chức năng cơ bản của hệ thống đồng hồ thời gian thực:

- Làm sáng lần lượt tất cả các led 7 thanh theo thứ tự từ trái sang phải.
- Làm sáng một năm bất kỳ trên led 7 thanh.
- Làm sáng tháng bất kỳ lên led 7 thanh.
- Làm sáng ngày bất kỳ lên led 7 thanh.
- Làm sáng giờ bất kỳ lên led 7 thanh.
- Làm sáng phút bất kỳ lên led 7 thanh.
- Làm sáng giây bất kỳ lên led 7 thanh.
- Làm sáng nhiệt độ được tính lên led 7 thanh.

Kiểm thử tất cả các led chạy năm, tháng, ngày, giờ, phút, giây theo thời gian thực được lấy từ IC thời gian thực DS1307.

5.2 Đánh giá hệ thống

Sau khi chạy mô phỏng trên phần mềm proteus và kiểm thử, em nhận thấy hệ thống đã đạt những yêu cầu của đề tài đã giao. Tuy nhiên vẫn còn chưa được ngắn gọn trong phần code. Dưới đây là đánh giá chi tiết về hệ thống:

- Độ chính xác hiển thị: led 7 thanh hiển thị đúng số cần hiển thị chạy lần lượt từ trái qua phải.
- Dữ liệu đầu vào đều hoạt động một cách chính xác.

Hiệu suất:

- Tốc độ làm mới số hiển thị thời gian đồng hồ thời gian thực rất ổn định.
- Thời gian phản hồi nhanh.

Độ bền bỉ:

- Có thể hoạt động liên tục trong thời gian dài và dễ sửa đổi.

- Khi mất nguồn điện và sau khi khôi phục nguồn điện đồng hồ thời gian thực vẫn chạy một cách trơn chu không cần cài lại.
- Dễ dàng sử dụng thay đổi cài đặt giờ chỉ bằng nút bấm.
- Nhiệt độ được hiển thị trên led tùy vào tính toán qua mạch.

Tóm lại, hệ thống đồng hồ dương lịch đã hoạt động dung như yêu cầu của đề tài đưa ra.

Hiển thị năm, tháng, ngày, giờ, phút, giây theo thời gian thực, nhiệt độ trên led 7 thanh.

KẾT LUẬN

Trong quá trình thực hiện đề tài "đồng hồ thời gian thực",tôi đã tập trung vào việc thiết kế ,mô phỏng và kiểm tra hệ thống trên môi trường mô phỏng. Dưới đây là những kết luận đưa ra từ dự án.

Thiết kế và mô phỏng hệ thống:

Hệ thống led 7 thanh được thiết kế với các thành phần cơ bản như PIC16F877A, IC DS1307 và một số linh kiện hỗ trợ khác.

Sơ đồ nguyên lý gồm khối nguồn, khối tín hiệu đầu vào, khối xử lý trung tâm, khối hiển thị.

Lập trình và điều khiển :

Chương trình điều khiển đã được viết và biên dịch bằng phần mềm Pic C CCS Compiler. Các đoạn mã nguồn chính bao gồm khai báo biến, hàm điều khiển LED 7 đoạn, và xử lý tín hiệu đầu vào từ người dùng.

Mã nguồn đã được tối ưu hóa để hoạt động hiệu quả trên môi trường mô phỏng, đảm bảo các chức năng của hệ thống hoạt động đúng như mong đợi.

Kiểm thử và đánh giá mô phỏng :

Hệ thống đã được kiểm thử kỹ lưỡng trong môi trường mô phỏng, bao gồm các bài kiểm tra chức năng và hiệu suất. Kết quả cho thấy hệ thống hoạt động ổn định, hiển thị chính xác các ký tự và hình ảnh theo yêu cầu.

Một số lỗi nhỏ đã được phát hiện và khắc phục trong quá trình kiểm thử, đảm bảo tính ổn định và độ tin cậy của hệ thống trong môi trường mô phỏng.

Đánh giá và cải tiến :

Hệ thống hiển thị led 7 đoạn đạt được mục tiêu như dự kiến ban đầu trong môi trường mô phỏng, cung cấp phương tiện hiển thị thông tin đơn giản và hiệu quả.

Tuy nhiên, hệ thống có thể hiển thị rõ nét hơn nếu được gán vào LCD bởi led 7 đoạn còn nhiều hạn chế khi hiển thị.

Hướng phát triển :

Thực hiện mạch thực tế : Chuyển từ mô phỏng sang xây dựng mạch thực tế để kiểm chứng các thiết kế và chương trình đã xây dựng.

Mở rộng kích thước LED 7 đoạn hay thay bằng LCD để hiển thị được nhiều ký tự và hình ảnh hơn.

Cải thiện độ sáng : Sử dụng các loại LED sáng hơn hoặc công nghệ LED mới để cải thiện khả năng hiển thị trong điều kiện ánh sáng mạnh.

Tích hợp kết nối Không dây : Bổ sung các module kết nối không dây như Bluetooth hoặc Wi-Fi để điều khiển hệ thống từ xa qua các thiết bị di động.

Phát triển ứng dụng điều khiển : Xây dựng các ứng dụng di động hoặc phần mềm PC để dễ dàng thay đổi nội dung hiển thị và điều khiển hệ thống.

Tóm lại, dự án "đồng hồ dương lịch" đã đạt được những kết quả đáng khích lệ trong môi trường mô phỏng, đáp ứng các yêu cầu kỹ thuật và mục tiêu đặt ra. Em hy vọng rằng những cải tiến trong tương lai sẽ làm cho hệ thống này trở nên hoàn thiện và hữu ích hơn trong nhiều ứng dụng thực tế.

TÀI LIỆU THAM KHẢO

1. **Microchip Technology Inc.**
   PIC16F877A Data Sheet, Microchip, USA.
   (Tài liệu chính thức về cấu trúc, thanh ghi, I/O, Timer, ADC của PIC16F877A)
2. **Maxim Integrated**
   DS1307 – Real-Time Clock (RTC) Data Sheet, Maxim Integrated.
   (Tài liệu chuẩn về IC thời gian thực DS1307, giao tiếp I2C, thanh ghi thời gian)
3. **Texas Instruments**
   LM35 Precision Centigrade Temperature Sensors, Texas Instruments.
   (Tài liệu cảm biến nhiệt độ LM35 dùng trong đo và hiển thị nhiệt độ)
4. **NXP Semiconductors**
   74HC595 – 8-bit Shift Register with Output Latches, NXP.
   (Tài liệu IC dịch nối tiếp–song song dùng mở rộng chân điều khiển LED 7 đoạn)
5. **Muhammad Ali Mazidi, Rolin D. McKinlay, Danny Causey**
   PIC Microcontroller and Embedded Systems – Using Assembly and C for PIC18, Pearson Education.
   (Giáo trình vi điều khiển PIC và hệ thống nhúng)
6. **John B. Peatman**
   Design with PIC Microcontrollers, Prentice Hall.
   (Thiết kế hệ thống nhúng sử dụng vi điều khiển PIC)
7. **Scott MacKenzie**
   The 8051 Microcontroller, Prentice Hall.
   (Tài liệu tham khảo thêm về nguyên lý vi điều khiển và hệ thống nhúng)
8. **Labcenter Electronics**
   Proteus Design Suite – User Guide.
   (Tài liệu mô phỏng mạch và chương trình trong Proteus)
9. **Nguyễn Văn Hiệp**
   Giáo trình Vi điều khiển PIC, NXB Khoa học & Kỹ thuật.
   (Tài liệu tiếng Việt hỗ trợ học và lập trình PIC)

Link github: https://github.com/vulan1112/k58ptp_baitap1_h-th-ng-nh-ng.git